

目录

1.STM32 启动方式?	2
2.STM32 上电到 main 函数之前做了什么事?	2
3.FreeRTOS 移植过程?	3
4.知道 volatile 关键字吗，解释一下	4
5.知道 inline 关键字吗，解释一下	5
6.信号量，互斥量的区别?	6
7.C 语言中的 static 做什么用的?	7
9.如何让程序陷入无限循环（C 语言）	8
11.已知有一个 int 型大小的内存及其地址，如何向该地址存入数据	9
13.进程的内存分配，按低地址到高地址系统地讲述一下	12
14.C 语言如何面向对象	15
16.IIC 了解吗?速率怎么样？时序和读写数据的过程?	16
18. 数组和指针有什么联系?	18
19. 堆和栈的区别	19
20. 线程和进程的区别	20
18. 共享内存的使用注意事项	20
20. 实现 strcpy 和 swap	24
21. 中断问题	24
21. 栈溢出是什么？为什么会？什么情况下会?	25
14.看门狗是什么？为什么要看门狗?	25
15.FreeRTOS 任务切换过程（内核）	错误！未定义书签。

1.STM32 启动方式?

<https://blog.csdn.net/ViatorSun/article/details/103771546>

<https://www.jianshu.com/p/4e606a5ac4bf>

3 种

1. 从 Flash 启动，将 Flash 地址 0x0800 0000 映射到 0x00000000，这样启动以后就相当于从 0x08000000 开始的，这是我们最常用的模式；
2. 从 SRAM 启动，将 SRAM 地址 0x20000000 映射到 0x00000000，这样启动以后就相当于从 0x20000000 开始的，用于调试，笔者基本没用过
3. 从系统存储器启动(System memory)，将系统存储器地址 0x1FFFF000 映射到 0x00000000，这样启动以后就相当于从 0x1FFFF000 开始执行的，值得注意的是这个系统存储器里面存储的其实是 STM32 自带的 Bootloader 代码，这其实是一个官方的 IAP，它提供了可以通过 UART1 接口将用户的代码下载到 Flash 中的功能，然后将 boot0 置 0，复位单片机，便可以 Flash 启动。

注：系统内存存储器我们没办法烧些，因为他是一个只读的 ROM，我们只能读取，但是可以通过从系统存储器启动，通过串口 UART1 烧录代码。

启动模式选择引脚		启动模式	说明
BOOT1	BOOT0		
X	0	主闪存存储器	主闪存存储器被选为启动区域
0	1	系统存储器	系统存储器被选为启动区域
1	1	内置SRAM	内置SRAM被选为启动区域

2.STM32 上电到 main 函数之前做了什么事?

<https://zhuanlan.zhihu.com/p/98888285>

```
; Reset handler
Reset_Handler PROC :定义函数
    EXPORT Reset_Handler [WEAK]
    IMPORT __main :引入函数
    IMPORT SystemInit :引入函数
    LDR R0, =SystemInit :将函数放到 R0
    BLX R0 :执行
    LDR R0, =__main
```

BX R0

ENDP

- 1、初始化堆栈指针 SP=_initial_sp，启动方式不同，指向地址不同。
- 2、初始化 PC（通用寄存器指针）指针=Reset_Handler，如果是 Flash 启动就是 0x0800 0000

表 7.6 上电后的向量表

地址	异常编号	值（32 位整数）
0x0000_0000	-	MSP 的初始值
0x0000_0004	1	复位向量（PC 初始值）
0x0000_0008	2	NMI 服务例程的入口地址
0x0000_000C	3	硬 fault 服务例程的入口地址
...	...	其它异常服务例程的入口地址

<https://viatorsun.blog.csdn.net/>

- 3、SystemInit（配置系统时钟）
- 4、__main(MDK 自带的函数)，初始化堆栈，.data .bss
- 5、最终调用 main 函数去到 C 的世界

3.FreeRTOS 移植过程？

（内核源码，头文件，内存，接口）

下载源码

拉取 FreeRTOSv9.0.0\FreeRTOS\Source 的所有.c 文件==>内核源码

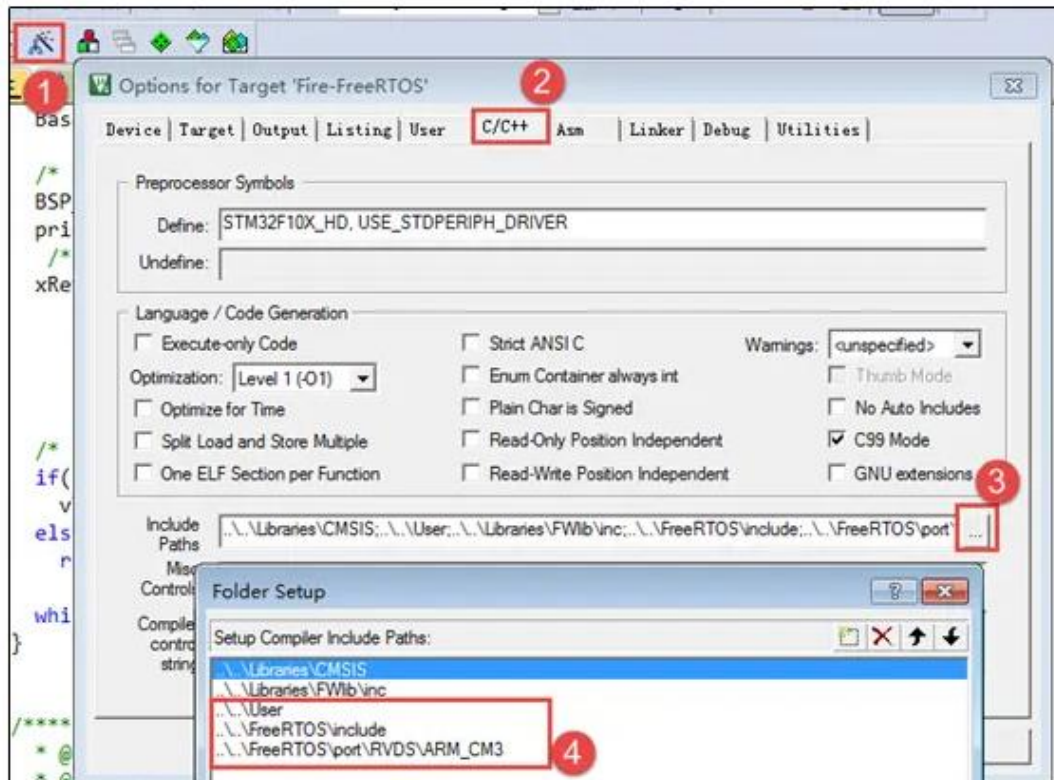
拉

取

FreeRTOSv9.0.0\FreeRTOS\Source\portable\{MemMang,\RVDS\ARM_CM3}

拉取 FreeRTOSv9.0.0\FreeRTOS\Source\include==》头文件

工程中 C++配置目录



修改 **FreeRTOSConfig.h** 文件，堆栈空间问题，时间节拍

修改服务中断函数，stm32f10x_it.c, SysTick_Handler(void)

注释掉 **PendSV_Handler()**、**SVC_Handler()**函数

4.知道 **volatile** 关键字吗，解释一下

<https://www.runoob.com/?s=volatile>

volatile 可以保证对特殊地址的稳定访问（变量地址），

到地址去读取

告诉编译器不要优化

用处

中断服务程序中修改的变量需要加 **volatile**;

全局变量;

多任务环境下各任务间共享的标志应该加 `volatile`;

```
volatile int a=10;  
// 别的任务中中断处理，将 a 变成了 20; ，编译器并不知道  
int b=a; // 正常： b=20, b=10;
```

5.知道 `inline` 关键字吗，解释一下

https://blog.csdn.net/zqixiao_09/article/details/50877383

<https://www.runoob.com/w3cnote/cpp-inline-usage.html>

处理一些需要频繁操作的函数。

内联函数

没用

```
inline void Foo(int x, int y); // inline 仅与函数声明放在一起  
void Foo(int x, int y)  
{  
}
```

有用

```
void Foo(int x, int y);  
inline void Foo(int x, int y) // inline 与函数定义体放在一起  
{  
}
```

`inline` 是 C++ 引入的关键字

`inline` 只适合函数体内代码简单的函数使用（`switch while for`）

`inline` 是一种"用于实现的关键字"

内联是以代码膨胀（复制）为代价

如果函数体内出现循环，那么执行函数体内代码的时间要比函数调用的开销大，（不能有 `switch while`）

`inline` 只适合函数体内代码简单的函数使用

6.信号量，互斥量的区别？

信号量可以存储多个值 012345，非负整数；

互斥量只能存在 1 和 0；

Mutex 本质上说就是一把锁，互斥功能，提供对资源的独占访问，所以 **Mutex** 主要的作用是用于互斥。

信号量(**Semaphore**)，同步，有时被称为信号灯，是在多线程环境下使用的一种设施, 它负责协调各个线程, 以保证它们能够正确、合理的使用公共资源。

信号量例子

比如一个教室，里面 10 个座位，那么信号量就是 10，进去了 10 个人(10 个任务获取了 10 次信号量)，信号量变成 0，这个时候要进去（获取）就进不去了，需要等待（或试图等待），当有人从教室出来（释放信号量），信号量+1，等待的人（任务）就可以进去（获取）了。

互斥量例子

一个打印机，每次都只能一个人（任务）使用（访问），那么这个情况下就使用互斥量，当有人在用的时候（互斥量为 0），别人就不能用了，当用完之后（互斥量为 1），在等待的人就可以去用了

状态

- 创建
- 释放
- 获取
- 等待（死等，等多长时间）
- 销毁

7.C 语言中的 **static** 做什么用的？

作用在变量上，变量存储在静态区，变量作用域只在本文件中

全局变量（作用域从开始定义到文件结束）

局部变量，函数执行完不会释放，下次依然可以使用

存储在.data 或.bss 区域

作用在函数，函数作用域只在本文件中（准确从定义之处开始，到文件结尾处结束）

8.char 型的数据，怎么将其第三位置 0 或者置 1

char ，一个字节，8 位数据，0000 0000，置 0，或置 1，0x00；

```
#include "stdio.h"
int main()
{
    unsigned char c=0X07;
    //c|=0x01<<2; // 置 1
```

```

c&=~(0x01<<2); // 置 0
printf("%x\r\n",c);
return 0;
}

```

置 0

0x07==>0000 0111

需要与上 1111 1011，才能保证其他位数不变，第三位置 0

0x01==0000 0001; 0x01<<2==0000 0100; ~(0x01<<2)==1111 1011;

置 1

0x03==0000 0011

或上 0000 0100,

0x01<<2;得到 0000 0100

9.如何让程序陷入无限循环（c 语言）

```

// for 循环
for(;;);

// while 循环
while(1);

// do while
do{}while(1);

// goto 方式
state:
goto state;

```

10.int a = -20,unsigned int b = 6， a+b 是否大于 6

有符号和无符号运算，强制转换为无符号，所有 a+b 会变成(unsigned int)a + b
(unsigned int)a 就会相当于 无符号最大值-20，那么是一个非常大的值，这个值加

上 6，那么肯定是大于 6 的；

最后的值是 $2^{32-20+6}=4294967282$ ，肯定时大于 6

```
#include "stdio.h"
int main()
{
    int a=-20;
    unsigned b=6;
    printf("\r\n%d,%u,%u,-1=%u\r\n", (a+b)>6, (unsigned int)a, (a+b), -1);
    return 0;
}
```

11. 已知有一个 int 型大小的内存及其地址，如何向该地址存入数据

内存，4 个字节，32 位

0x3a47ba14;

```
#include "stdio.h"

int main()
{

    int *p=(int *)0x3a47ba14; // 地址
    *p=10
    printf("%d,0x%x", *p, p); // 输出地址的信息

    return 0;
}
```

12. 进程通信的方式？共享内存是怎么做的？

如果两个进程同时读写这块内存怎么办？

进程通信方式：

信号量

消息队列

管道

匿名（需要血缘关系）

有名（不需要血缘关系）

高级（另一个程序在当前程序打开，算是当前程序的子进程，称高级管道）

socket（套接字）

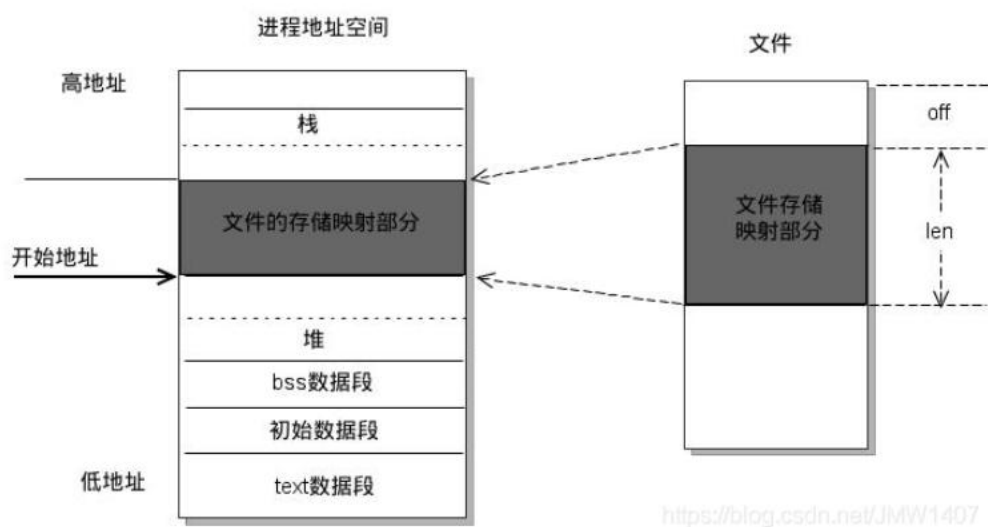
共享内存通信（最快）

共享内存怎么做？

特别提醒：共享内存并未提供同步机制，也就是说，在第一个进程结束对共享内存的写操作之前，并无自动机制可以阻止第二个进程开始对它进行读取。所以我们通常需要用其他的机制来同步对共享内存的访问，例如信号量。

1. 创建共享内存，指定 key、size、mode（权限）
2. 将当前进程连接到共享内存
3. 把共享内存的地址读取出来，并强制转换为本地变量
4. 读写操作了

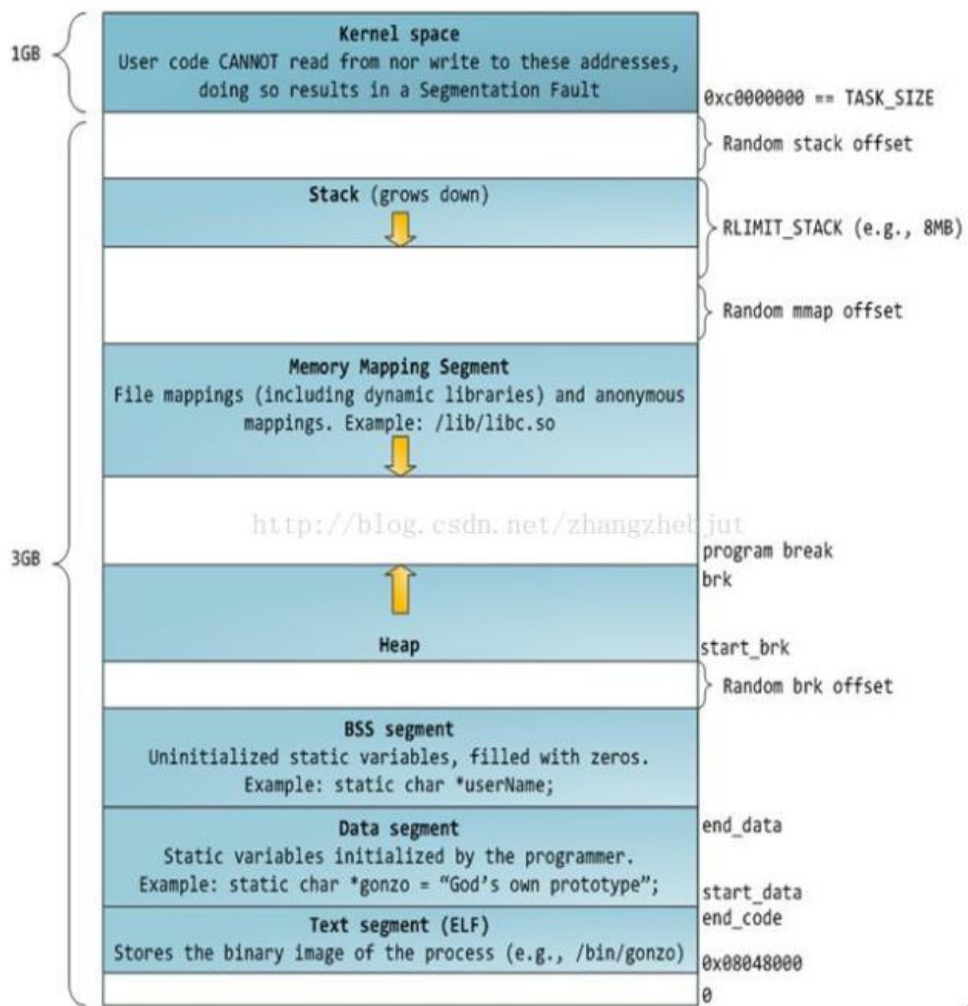
做法：将同一段物理内存映射到堆与栈之间的那一段虚拟内存中，



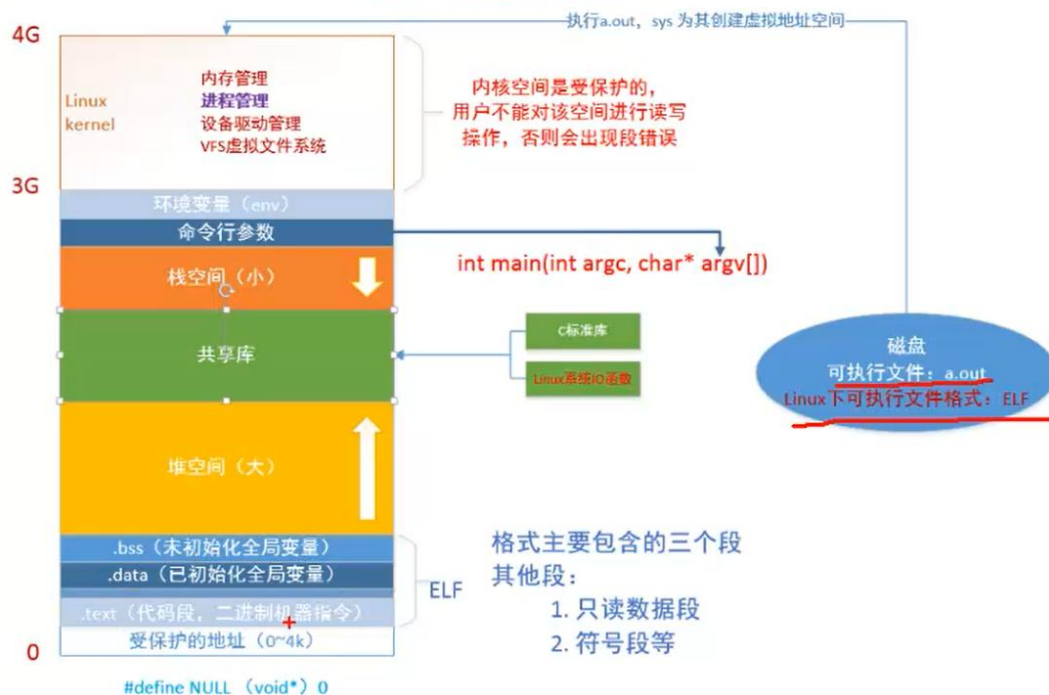
<https://blog.csdn.net/ljianhui/article/details/10253345>

13.进程的内存分配，按低地址到高地址系统地讲述一下





Linux 每一个运行的程序（进程）操作系统都会为其分配一个 0~4G 的地址空间（虚拟地址空间）
进程：正在运行的程序



分配 4G 的虚拟空间，用多少占用多少，然后 1G 内核。

进程的内存空间从低地址到高地址内存布局依次是：保留区 → 文本段（包含程序和字符串常量）→ 初始化的变量（.data，包含全局变量和静态变量）→ 未初始化的变量（.bss，包含全局变量和静态变量）→ 堆 → 共享库或 mmap → 栈 → 命令函数参数（main 函数的参数）→ 环境变量 → 内核空间

栈向低地址推

堆向高地址推

malloc 的时候（堆）

brk，将堆的最高地址往高出推，只能高地址释放，低地址才能释放

mmap，随便释放，找一块空闲的虚拟内存

虚拟内存解决了一个物理空间不连续问题，将断断续续的物理内存映射到虚拟内存上，起到看起来是连续的一段空间。

1、brk 是将堆的最高地址指针 `_edata` 往高地址推；

2、mmap 是在进程的虚拟地址空间中（堆和栈中间，称为文件映射区域的地方）找一块空闲的虚拟内存。

这两种方式分配的都是**虚拟内存**，没有分配物理内存。在第一次访问已分配的虚拟地址空间的时候，发生**缺页中断**，操作系统负责分配物理内存，然后建立虚拟内存和物理内存之间的映射关系。

缺页中断===》 一页=4096 字节=4KB（注意是 Byte，1B=8bit）

14.C 语言如何面向对象

多态

```
#include "stdio.h"

typedef struct Person{
    char *name;
    void (*eat)();
}Person;

void fun1(){
    printf("student eat!\r\n");
}
void fun2(){
    printf("teacher eat!\r\n");
}
int main(){
    Person student={"student",fun1};
    Person teacher={"teacher",fun2};

    student.eat();
    teacher.eat();
}
```

15.const 用法

const 右边靠近谁，谁就不可变，**靠近*就是指向的不可变**，**靠近变量名**，就是该变量不可变

```
const int *p; //const 修饰*p,p 是指针，*p 是指针指向的对象，不可变
int const *p; //const 修饰*p,p 是指针，*p 是指针指向的对象，不可变
int *const p; //const 修饰 p, p 不可变，p 指向的对象可变
const int *const p; //前一个 const 修饰*p,后一个 const 修饰 p, 指针 p 和 p 指向的对象
```

16.IIC 了解吗?速率怎么样? 时序和读写数据的过程?

IIC 总线协议定义如下:

- 只有在总线不忙时才能启动数据传输。

总线空闲

在停止条件之后、启动条件之前的时间内，数据线和时钟线都保持高电平。

在数据传输期间，只要 SCLx 时钟线为高电平，数据线就必须保持稳定。

在 SCLx 时钟线为高电平时，数据线的电平变化将被解析为启动或停止条件。

从器件应答 (ACK、NACK)

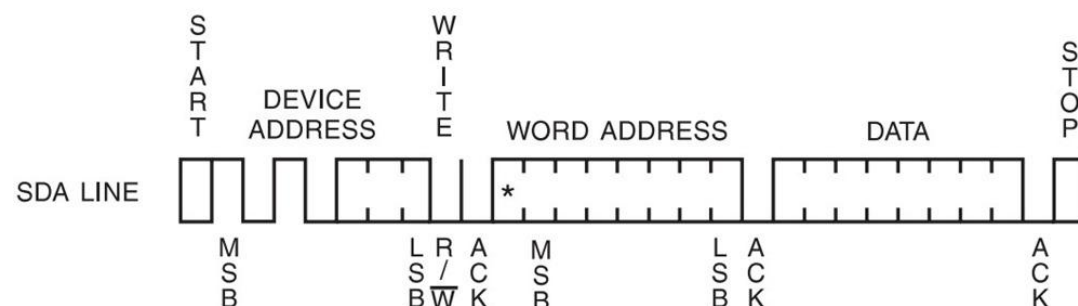
所有的数据字节传输都必须由接收器应答 (ACK) 或不应答 (NACK)。接收器将 SDAx 线拉低则发出 ACK，释放 SDAx 线则发出 NACK。应答为一位周期，使用一个 SCLx 时钟。

标准模式 I²C 总线规范在 80 年代的初期已经存在，它规定数据传输速率可高达 100kbit/s 而且 7 位寻址。这个概念在普及中迅速成长，今天它已经作为一个标准被全世界接受，而且 Philips Semiconductors 和其他供应商提供了几百种不同的兼容 IC。为了符合更高速度的要求以及制造更多可使用的从机地址给数量不断增长的新器件，标准模式 I²C 总线规范不断升级，到今天它提供了以下的扩展：

- 快速模式，位速率高达 400kbit/s。
- 高速模式 (Hs 模式)，位速率高达 3.4Mbit/s。
- 10 位寻址，允许使用高达 1024 个额外的从机地址。

<https://zhuanlan.zhihu.com/p/34674402>

主机写的时序



产生 start 位

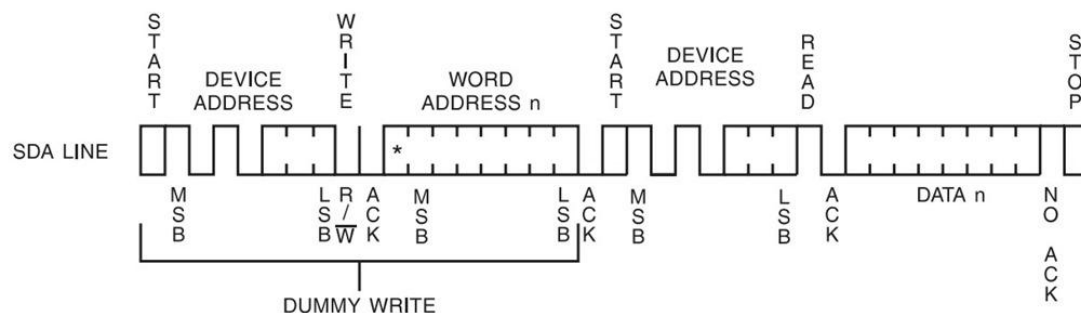
传送器件地址 ID_Address，器件地址的最后一位为数据的传输方向位，R/W，低电平 0 表示主机往从机写数据（W），1 表示主机从从机读数据（R）。ACK 应答，应答是从机发送给主机的应答，这里不用管。

传送写入器件寄存器地址，即数据要写入的位置。同样 ACK 应答不用管。

传送要写入的数据。ACK 应答不用管。

产生 stop 信号。

主机读的时序：



产生 start 信号

传送器件地址（写 ID_Address），ACK。

传送字地址（写 REG_Address），ACK。

再次产生 start 信号

再传送一次器件地址，ACK。

读取一个字节的数据，读数据最后结束前无应答 ACK 信号。

产生 stop 信号。

总结

启示信号：SCL 高，SDA 出现下降沿

结束信号：SCL 高，SDA 出现上升沿

应答信号：在数据传输后(8 位)，从机拉低 SDA，则为应答 ACK

写：在地址（7 位）后加上 0，发三（从机地址，寄存器地址，数据）应答三；

读：先假写（7 位后+0），再读（7 位后+1），启动两次，发三（从机地址，寄存器地址，从机地址）读一（读就行了）应答 3，最后没有应答，从机写完数据之后就释放总线。

17. 开漏输出和推挽输出的区别？IIC 为什么

一定要用开漏输出？

开漏输出和推挽输出的区别？

<https://zhuanlan.zhihu.com/p/41942876>

推挽输出的最大特点是可以真正能真正的输出高电平和低电平，在两种电平下都具有驱动能力。

常说的与推挽输出相对的就是开漏输出，对于开漏输出和推挽输出的区别最普遍的说法就是开漏输出无法真正输出高电平，即高电平时没有驱动能力，需要借助外部上拉电阻完成对外驱动。

所谓的驱动能力，就是指输出电流的能力。

IIC 为什么一定要用开漏输出？

https://blog.csdn.net/qq_31073871/article/details/86593925

为什么 IIC 必须要接上拉？

换句话说，IIC 的从器件（如 24C04 等），不具备拉高总线的能力，从机控制 SDA 上的电平，只能通过令 SDA 接地或不接地来实现输出 0 或 1，而无法通过接地或接 VCC 来实现输出 0 或 1。这就要求 IIC 总线上必须是默认上拉的，否则从机无法令 SDA 为高。

从另一方面讲：只有开漏上拉，才能满足“线与”逻辑，也即 SDA 线上多个从机中的任何一个拉低了 SDA，就会导致 SDA 为 0。

主机为什么使用开漏？试想，

主机如果使用了推挽，当主机释放 SDA 时（令 SDA=1 时），相当于 SDA 被接通到 VCC，这时从机无法控制 SDA 为 0；

主机如果使用了开漏，当主机释放 SDA 时（令 SDA=1 时），相当于 SDA 被悬空上拉到 VCC，这时从机可以拉低 SDA，使 SDA=0，也可以释放 SDA 使 SDA=1，这样从机才能发数据。

IIC 的 SCL 和 SDA 都要求接上拉电阻，也即，只要主机和从机都还没开始干活，那么 SCL 和 SDA 就都是空闲的（默认的高电平状态被称为“空闲”，也可以称总线被“释放”），同理，SCL 或 SDA 为 low 时，就称总线被占用或正忙。

IIC 协议要求：SCL 和 SDA 必须都是【开漏+上拉】！

18. 数组和指针有什么联系？

指针就是指针，指针变量在 32 位系统下，永远占 4 个 byte，其值为某一个内存的地址。指针可以指向任何地方，但是不是任何地方你都能通过这个指针变量访问到。

数组就是数组，其大小与元素的类型和个数有关。定义数组时必须指定其元素的类型和个数。数组可以存任何类型的数据，但不能存函数。

以下标的形式访问在本质上与以指针的形式访问没有区别，只是写法上不同罢了

指针和数组根本就是两个完全不一样的东西。只是它们都可以“以间接访问（*）形式”或“以下标形式”进行访问。

一个是完全的匿名访问（指针），一个是典型的具名+匿名访问（数组）。

```
int a[5]={1,2,3,4,5};
```

19. 堆和栈的区别

《C 语言深度解剖》中的解释

栈：保存局部变量。栈上的内容只在函数的范围内存在，当函数运行结束，这些内容也会自动被销毁。其特点是效率高，但空间大小有限。

堆：由 malloc 系列函数或 new 操作符分配的内存。其生命周期由 free 或 delete 决定。在没有释放之前一直存在，直到程序结束。其特点是使用灵活，空间比较大，但容易出错。

参考

<https://blog.csdn.net/k346k346/article/details/80849966>

堆与栈实际上是操作系统对进程占用的内存空间的两种管理方式，主要有如下几种区别：

（1）管理方式不同。栈由操作系统自动分配释放，无需我们手动控制；堆的申请和释放工作由程序员控制，容易产生内存泄漏；

（2）空间大小不同。每个进程拥有的栈的大小要远远小于堆的大小。

（3）生长方向不同。堆的生长方向向上，内存地址由低到高；栈的生长方向向下，内存地址由高到低。

（4）分配方式不同。堆都是动态分配的，没有静态分配的堆。栈有 2 种分配方式：静态分配和动态分配。栈的分配系统会自动分配，用户也可以申请（通过 alloca）；

(5) 分配效率(速度)不同。栈有底层指令实现，速度快；堆使用 C 库函数运算来申请和管理，速度慢；

(6) 存放内容不同。栈存放的内容，函数返回地址（调用函数的指令的下一条指令）、相关参数、局部变量和寄存器内容等。注意静态变量是存放在数据段或者 BSS 段，是不入栈的。而堆中具体存放内容是由程序员来填充的。

20. 线程和进程的区别

<https://cloud.tencent.com/developer/article/1688297>

一个线程只能属于一个进程，而一个进程可以有多个线程，但至少有一个线程。线程依赖于进程而存在。

进程在执行过程中拥有独立的内存单元，而多个线程共享进程的内存。（资源分配给进程，同一进程的所有线程共享该进程的所有资源。同一进程中的多个线程共享代码段（代码和常量），数据段（全局变量和静态变量），扩展段（堆存储）。但是每个线程拥有自己的栈段，栈段又叫运行时段，用来存放所有局部变量和临时变量。）

进程是资源分配的最小单位，线程是 CPU 调度的最小单位；

系统开销：进程切换的开销也远大于线程切换的开销。

通信：由于同一进程中的多个线程具有相同的地址空间，致使它们之间的同步和通信的实现，也变得比较容易。进程间通信 IPC，线程间可以直接读写进程数据段（如全局变量）来进行通信——需要进程同步和互斥手段的辅助，以保证数据的一致性。

进程间不会相互影响；线程一个线程挂掉将导致整个进程挂掉

18. 共享内存的使用注意事项

共享内存就是通过两个或连个以上的进程共享同一块内存区域来实现进程间的通信。

存放在共享内存的数据，任何进程都可以直接对其进行读取和修改。

因此共享内存所实现的进程通信是最快的。

多个进程在同时读写某一块共享内存时，内存中的数据会混乱，需要借助其他进程通信方式（比如信号量），来进行进程间的同步。

19. malloc 底层原理

linux 一页的大小

<https://blog.csdn.net/lansky54/article/details/5414199>

获取 Linux 内存页（基页）大小的命令：getconf PAGE_SIZE，一般的输出是 4096，即 4KB。

页中断

https://blog.csdn.net/m0_37962600/article/details/81448553

一、什么是缺页中断？

进程线性地址空间里的页面不必常驻内存，在执行一条指令时，如果发现他要访问的页没有存在内存中（即存在位为 0），那么停止该指令的执行，并产生一个页不存在的异常，对应的故障处理程序可通过从外存加载该页的方法来排除故障，之后，原先引起的异常的指令就可以继续执行，而不再产生异常。

底层原理（正文）

<https://www.jianshu.com/p/7af62431983e>

<https://www.cnblogs.com/zpcoding/p/10808969.html>

结论

- 1) 当开辟的空间小于 128K 时，调用 brk（）函数，malloc 的底层实现是系统调用函数 brk（），其主要移动指针 _enddata(此时的 _enddata 指的是 Linux 地址空间中堆段的末尾地址，不是数据段的末尾地址)
- 2) 当开辟的空间大于 128K 时，mmap（）系统调用函数来在虚拟地址空间中（堆和栈中间，称为“文件映射区域”的地方）找一块空间来开辟。

内存分配的原理

从操作系统角度看，进程分配内存有两种方式，分别由两个系统调用完成：brk 和 mmap（不考虑共享内存）

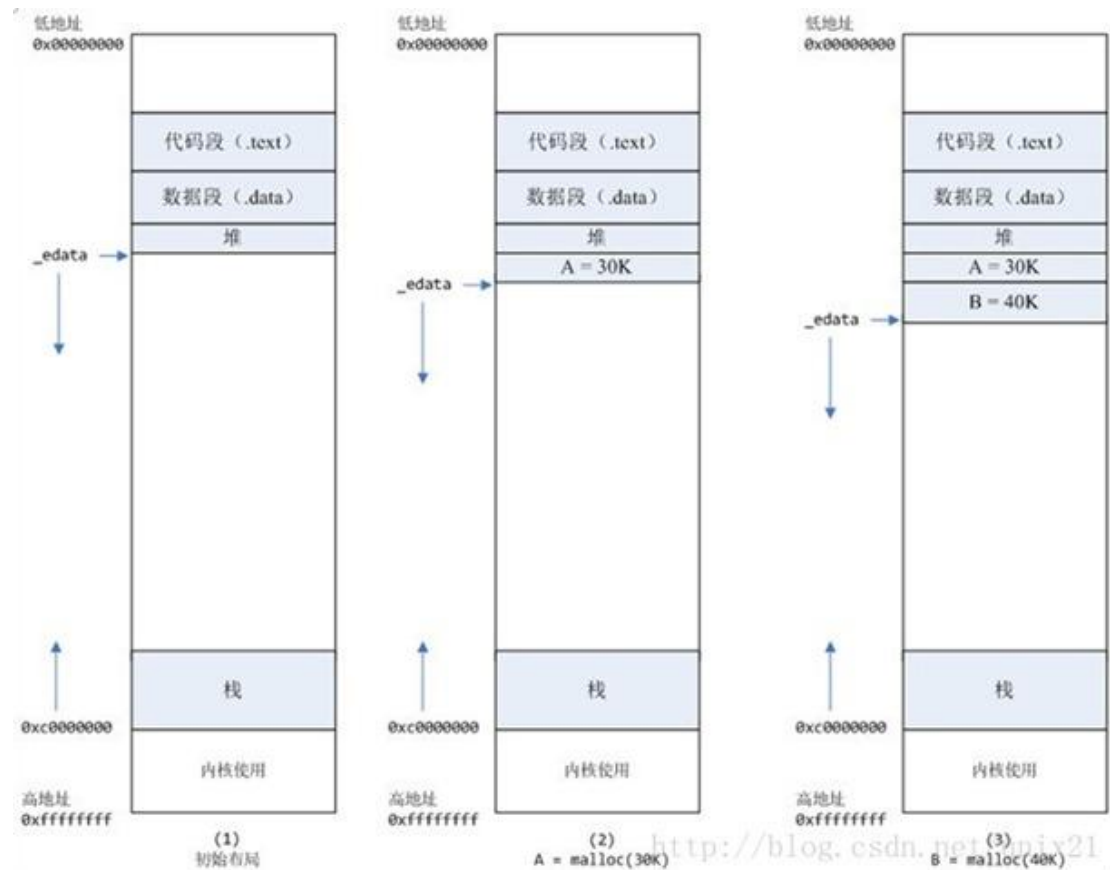
- 1) brk 是将数据段（.data）的最高地址指针 _edata 往高地址推
- 2) mmap 是在进程的虚拟地址空间中（堆和栈中间，称为“文件映射区域”的地方）找一块空闲的虚拟内存。

这两种方式分配的都是虚拟内存，没有分配物理内存。在第一次访问已分配的虚拟地址空间的时候，发生缺页中断，操作系统负责分配物理内存，然后建立虚拟内存和物理内存之间的映射关系。

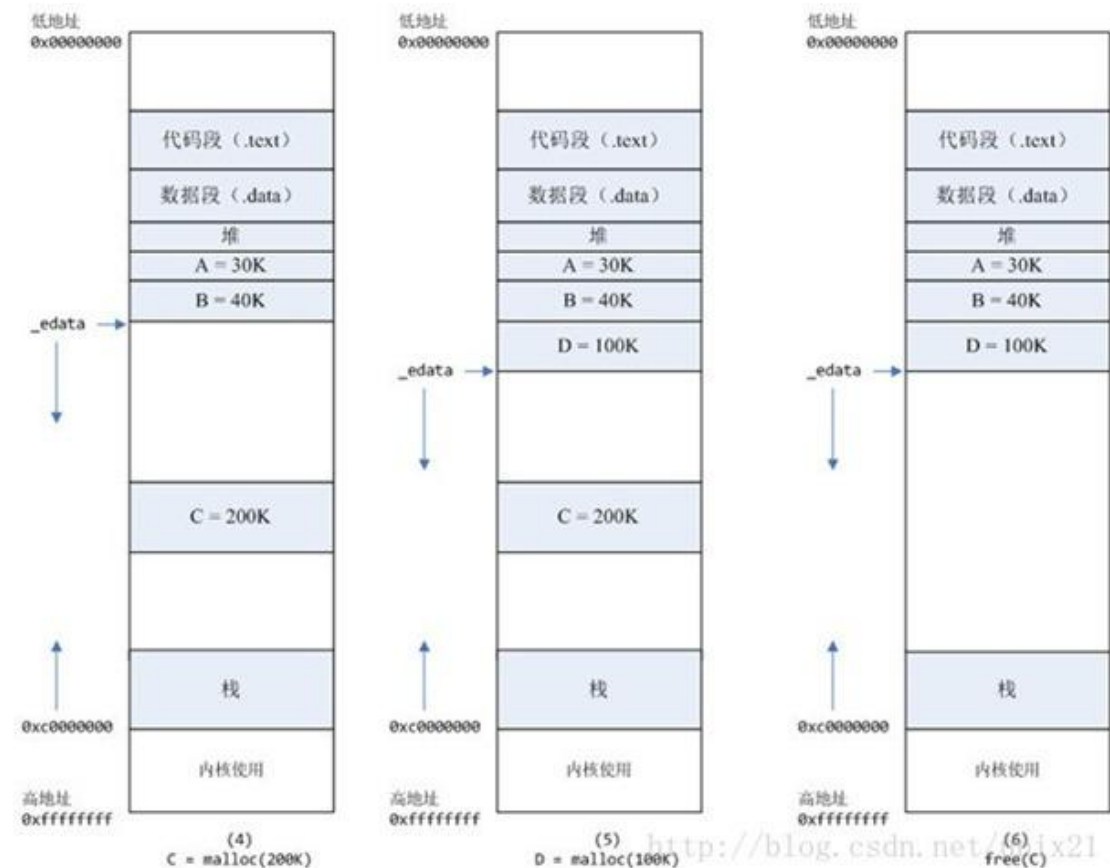
具体分配过程

情况一：malloc 小于 128K 的内存，使用 brk 分配

将 _edata 往高地址推（只分配虚拟空间，不对应物理内存（因此没有初始化），第一次读/写数据时，引起内核缺页中断，内核才分配对应的物理内存，然后虚拟地址空间建立映射关系），如下图：



情况二：malloc 大于 128K 的内存，使用 mmap 分配 (munmap 释放)



默认情况下，malloc 函数分配内存，如果请求内存大于 128K（可由 M_MMAP_THRESHOLD 选项调节），那就不是去推_edata 指针了，而是利用 mmap 系统调用，从堆和栈的中间分配一块虚拟内存

这样子做主要是因为：

brk 分配的内存需要等到高地址内存释放以后才能释放（例如，在 B 释放之前，A 是不可能释放的，因为只有一个_edata 指针，这就是内存碎片产生的原因，什么时候紧缩看下面），而 mmap 分配的内存可以单独释放。

进程调用 free(B) 以后

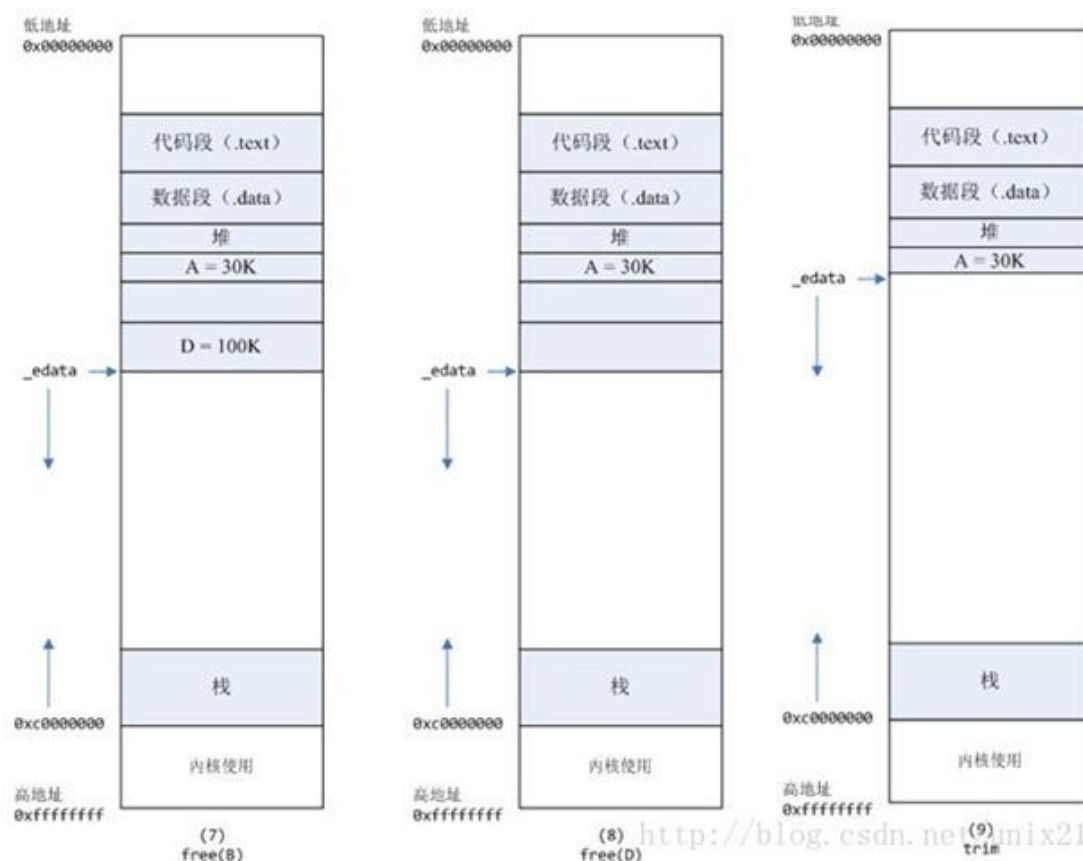
B 对应的虚拟内存和物理内存都没有释放，因为只有一个_edata 指针，如果往回推，那么 D 这块内存怎么办呢？当然，B 这块内存，是可以重用的，如果这个时候再来一个 40K 的请求，那么 malloc 很可能就把 B 这块内存返回回去了

进程调用 free(D) 以后，如图 8 所示

B 和 D 连接起来，变成一块 140K 的空闲内存

默认情况下：

当最高地址空间的空闲内存超过 128K（可由 M_TRIM_THRESHOLD 选项调节）时，执行内存紧缩操作（trim）。在上一个步骤 free 的时候，发现最高地址空闲内存超过 128K，于是内存紧缩。



20. 实现 strcpy 和 swap

```
#include "stdio.h"

void mstrcpy(char * dest,const char * src){
    while(*src!='\0'){
        *dest++=*src++;
    }
}

void swap(int *a,int *b){ // 5 6
    *a=*a+*b;// a=11
    *b=*a-*b;// b=11-6=5
    *a=*a-*b;// a=11-5=6
}

int main(){
    char buf[20]={0};
    mstrcpy(buf,"123123123123");
    printf("\r\n%s\r\n",buf);
    int a=10;
    int b=20;
    swap(&a,&b);
    printf("\r\n%d,%d\r\n",a,b);
    return 0;
}
```

21. 中断问题

<https://cloud.tencent.com/developer/article/1829273>

以下代码有什么错误？

```
_interrupt double compute_area (double radius)
{
    double area = PI * radius * radius;
    printf("\nArea = %f", area);
    return area;
}
```

21. 栈溢出是什么？为什么会？什么情况下会？

ioctl 的实现原理、内核态和用户态怎么交互

14.看门狗是什么？为什么要看门狗？

合并两个有序数组

```
#include "stdio.h"

void func(int * resultA,int *a1,int aC1,int *a2,int aC2){
    while (aC1-->0)
    {
        *resultA++=*a1++;
    }
    while (aC2-->0)
    {
        *resultA++=*a2++;
    }
}
```

```

void sort(int *arr,int len){
    for (int i = 0; i < len; i++)
    {
        for (int j = 0; j < len-1; j++)
        {
            if(arr[j]>arr[j+1]){
                int temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
}

int main(){
    int a[10]={};
    int b[5]={1,2,3,46,7};
    int c[5]={0,2,5,6,1};
    // printf("%d\r\n",sizeof(c)/sizeof(int));
    func(a,b,5,c,5);
    for (int i = 0; i < 10; i++)
    {
        printf("%d,",a[i]);
    }

    printf("\r\n");
    sort(a,10);
    for (int i = 0; i < 10; i++)
    {
        printf("%d,",a[i]);
    }
    printf("\r\n");
}

```

